DVSA

Vulnerability Discovery & Remediation

ICS 344 – Information Security

Section F12

Danah Alsawad · Jana Alkahlan

2 May 2026 | us-east-1

10 Official Vulnerability Lessons

- #01 Event Injection & Vuln. Dependencies
- #02 Broken Authentication
- #03 Sensitive Information Disclosure
- #04 Insecure Cloud Configuration
- #05 Broken Access Control
- #06 Denial of Service (DoS)
- #07 Over-Privileged Function
- #08 Logic Vulnerabilities
- #09 Vulnerable Dependencies
- #10 Unhandled Exceptions

#01

Event Injection & Vulnerable Dependencies

Affected: DVSA-ORDER-MANAGER Lambda / node-serialize package

Exploit

- Sent malicious payload with $$ND_FUNC$$ marker via /order API
- node-serialize evaluated & executed injected JS on deserialization
- Arbitrary code ran before any application logic
- CloudWatch confirmed LAMEES_LESSON_1_SUCCESS – RCE proven

Fix Applied

- Replaced serialize.unserialize() with JSON.parse() in order-manager.js
- JSON.parse() treats input as data only – never executes functions
- Added input allowlist for the 'action' field
- Added $$ND_FUNC$$ pattern detection to block payloads early

Verified ✓

- Replayed identical injection payload after fix
- CloudWatch shows only START, END, REPORT – no code execution
- No LAMEES_LESSON_1_SUCCESS log entry appears
- Normal order requests still succeed (200 OK)

#02

Broken Authentication

Affected: JWT handling in DVSA-ORDER-MANAGER / get_orders Lambda

Exploit

- Captured JWT tokens for User B (attacker) and User C (victim) via DevTools
- Decoded payload with Python; swapped username/sub with victim's ID
- Re-encoded token with no valid signature – backend accepted it
- API returned User C's full order history – account takeover confirmed

Fix Applied

- Added verifyCognitoJwt() using JWKS public keys from Cognito
- Verifies RS256 signature before trusting any JWT claims
- Validates iss (issuer) and exp (expiry) fields after verification
- User identity only extracted from successfully verified tokens

Verified ✓

- Replayed forged token after fix
- API returns {"status":"err","msg":"invalid token"}
- Access denied – victim's orders no longer accessible
- Legitimate valid tokens continue to work correctly

#03

Sensitive Information Disclosure

Affected: DVSA-ORDER-MANAGER → internal DVSA-ADMIN-GET-RECEIPT Lambda

Exploit

- Injected JS via $$ND_FUNC$$ to invoke DVSA-ADMIN-GET-RECEIPT Lambda
- Payload exfiltrated result to external Webhook.site listener
- Webhook received signed S3 URL containing all users' order receipts
- Downloaded 2026-04-dvsa-order-receipts.zip – full data breach

Fix Applied

- Replaced serialize.unserialize() with JSON.parse() – eliminates RCE
- Added $$ND_FUNC$$ early detection for immediate request rejection
- Action whitelist prevents calls to internal/admin functions
- Strengthened admin check: strict equality (===) enforced

Verified ✓

- Replayed injection payload targeting DVSA-ADMIN-GET-RECEIPT
- API returns {"status":"err","message":"Malicious input detected"}
- No webhook activity; no S3 URL generated; no file downloaded
- Normal authenticated requests continue to work correctly

#04

Insecure Cloud Configuration

Affected: dvsa-feedback-bucket S3 Bucket Policy (public write)

Exploit

- Bucket policy showed "Principal": "*" with s3:PutObject allowed
- Uploaded exploit.raw with no credentials: aws s3 cp --no-sign-request
- Upload succeeded – unauthenticated file stored in application bucket
- CloudWatch logged ObjectCreated:Put for attacker-controlled file

Fix Applied

- Removed "Principal": "*" from bucket policy entirely
- Restricted Principal to account ARN only: arn:aws:iam::433...:root
- Removed s3:PutObjectAcl from allowed actions
- Only authenticated IAM identities can now write to the bucket

Verified ✓

- Repeated aws s3 cp exploit.raw ... --no-sign-request after fix
- Upload fails: An error occurred (AccessDenied)
- Unauthorized uploads completely blocked
- Authorized account access still functions normally

#05

Broken Access Control

Affected: DVSA-ORDER-MANAGER → DVSA-ADMIN-UPDATE-ORDERS

Exploit

- Placed normal order (status: processed), captured JWT
- Crafted $$ND_FUNC$$ payload to invoke DVSA-ADMIN-UPDATE-ORDERS
- Payload set order status = 120 (paid) without any billing
- Order status changed to 'paid' – financial manipulation confirmed in UI

Fix Applied

- Replaced unsafe deserialize with JSON.parse() – eliminates RCE entry
- Added Cognito AdminGetUser check – only admin role triggers updates
- Safe Lambda invocation using InvokeCommand with typed payload
- Regular users are fully blocked from reaching admin-only functions

Verified ✓

- Replayed exploit payload after fix
- Response: {"status":"err","msg":"Invalid JSON"} – payload rejected
- Order status remains 'processed'; no unauthorized state change
- Admin-triggered order updates still function as expected

#06

Denial of Service (DoS)

Affected: API Gateway billing endpoint – no rate limiting configured

Exploit

- Python script sent hundreds of concurrent billing requests via threading
- Backend saturated – responses degraded to 500 / 502 errors
- Script hit RuntimeError: can't start new thread – client also overwhelmed
- Legitimate billing requests blocked – service made unavailable

Fix Applied

- Enabled throttling on API Gateway Stage (dvsa)
- Rate limit: 5 requests per second
- Burst limit: 2 concurrent requests
- Excess requests immediately returned 429 Too Many Requests

Verified ✓

- Reran DoS script after applying throttling
- All flood requests return 429 {"message":"Too Many Requests"}
- Backend remains stable – no 500/502 errors under flood
- Single request within rate limit completes successfully

#07

Over-Privileged Function

Affected: IAM Execution Role of DVSA-ORDER-MANAGER (AmazonCognitoPowerUser)

Exploit

- Used $$ND_FUNC$$ injection to log process.env to CloudWatch
- Extracted AWS_ACCESS_KEY_ID, SECRET_ACCESS_KEY, SESSION_TOKEN
- Confirmed stolen identity with: aws sts get-caller-identity
- aws cognito-idp list-users returned all user emails & phone numbers

Fix Applied

- Removed AmazonCognitoPowerUser policy from Lambda execution role
- Replaced with minimal policy: logs:CreateLogGroup/Stream/PutLogEvents only
- Lambda execution role has zero Cognito permissions now
- Principle of Least Privilege enforced on the IAM role

Verified ✓

- Reused stolen STS credentials after IAM policy change
- aws cognito-idp list-users returns AccessDeniedException
- Stolen credentials can no longer access Cognito user data
- Lambda still logs correctly – core functionality preserved

#08

Logic Vulnerabilities (Race Condition)

Affected: DVSA-ORDER-UPDATE Lambda / orderStatus boundary check

Exploit

- Added 1 item to cart; captured JWT + order ID at billing page
- Sent billing request and qty-update (→5) simultaneously (0.1s apart)
- ORDER-UPDATE allowed update: orderStatus (100) was NOT > 110
- DynamoDB: 5 items recorded, totalAmount charged for only 1 – financial loss

Fix Applied

- Changed condition in update_order.py (DVSA-ORDER-UPDATE Lambda)
- Before: if orderStatus > 110
- After: if orderStatus >= 100
- Update now blocked the moment billing starts (status set to 100)

Verified ✓

- Replayed simultaneous billing + update attack after fix
- Update response: {"status":"err","msg":"order already paid"}
- Item quantity remains 1; total amount is correct
- Normal pre-billing updates still work as expected

#09

Vulnerable Dependencies

Affected: node-serialize npm package in DVSA-ORDER-MANAGER

Exploit

- node-serialize executes any function embedded with $$ND_FUNC$$ marker
- Payload used fs module to write & read /tmp/pwned.txt inside Lambda
- CloudWatch: FILE READ SUCCESS: You are reading the contents of my hacked file!
- Full RCE inside Lambda despite API returning 500 externally

Fix Applied

- Removed require('node-serialize') from order-manager.js entirely
- Replaced unserialize() with JSON.parse() + raw event.headers
- Added action type-check: typeof req.action !== 'string' → 400
- Authorization header check added as first processing guard

Verified ✓

- Replayed $$ND_FUNC$$ payload after removing node-serialize
- Response: {"status":"err","message":"Missing auth"} – blocked at auth
- No FILE READ SUCCESS in CloudWatch; no /tmp/pwned.txt created
- Valid authenticated requests are processed normally

#10 Unhandled Exceptions

Affected: DVSA-ORDER-MANAGER – no error sanitization on Lambda responses

Exploit

- Sent incomplete request: {"action":"shipping"} with no order-id field
- Lambda threw KeyError on missing field – unhandled exception propagated
- Response exposed: full stackTrace, internal path /var/task/order_shipping.py:18
- errorType, errorMessage, and code references leaked directly to client

Fix Applied

- Wrapped Lambda invocation response in try/catch block
- Detects errorMessage, errorType, stackTrace fields in response payload
- Returns generic: {"status":"error","message":"Something went wrong"}
- Full error details logged to CloudWatch only – never sent to client

Verified ✓

- Replayed malformed request after adding error sanitization
- Response: {"status":"error","message":"Something went wrong"}
- No stack traces, file paths, or code references in client response
- Valid requests with all required fields still return correct results

Key Takeaways

10 lessons · 10 exploits · 10 fixes · all verified

01 Never trust user input as code

Replace unsafe deserializers (node-serialize) with JSON.parse(). Treat all input strictly as data.

02 Always verify JWTs server-side

Validate signatures against JWKS. Never decode-and-trust a token without cryptographic verification.

03 Enforce least privilege on IAM roles

Lambda execution roles must only contain the exact permissions required for their specific task.

04 Rate-limit at the API edge

API Gateway throttling prevents DoS. Backend Lambda concurrency limits alone are insufficient.

05 Lock order state atomically

Block updates the moment billing starts (orderStatus >= 100, not > 110). Timing gaps cause race conditions.

06 Sanitize all error responses

Log full details to CloudWatch only. Return only generic messages to clients – never expose stack traces.